Desenvolvimento Web

Profª Sabrina B. Moreira

Introdução a arquitetura Cliente-Servidor

Aula 2

Cliente-Servidor: histórico

- A **computação centralizada** foi predominante nas décadas de 1960 e 1970.
- Criou as bases para a computação moderna, inclusive a computação em nuvem.
- Principais características da computação centralizada são:

Cliente-Servidor: histórico

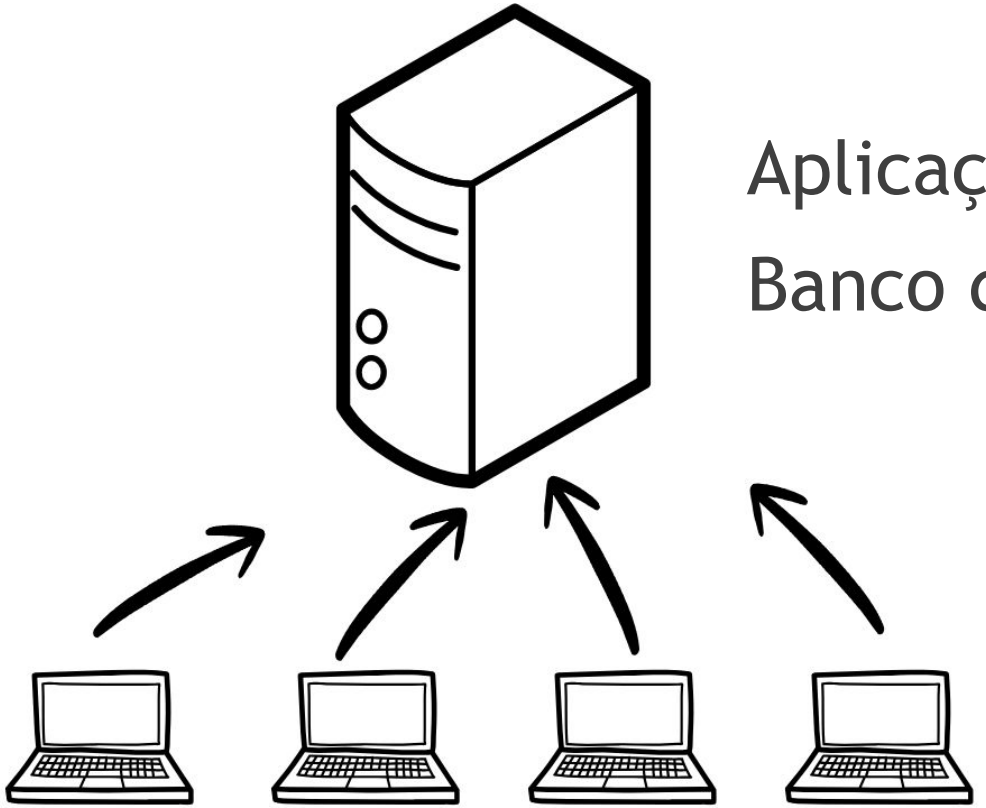
- Todo o processamento acontecia somente em um computador central, o **mainframe**.
- Esse computador era acessado via "**terminais burros**", que eram somente monitor e teclado.
- Além do processamento, o armazenamento e controle ficavam em um único ponto.

Cliente-Servidor: “terminal burro”

Fonte: technologyuk, 2026

- Não possui processamento próprio relevante
- Não executa lógica de aplicação
- Apenas envia entrada (teclado)
- Apenas exibe saída (monitor)

Cliente-Servidor: modelo centralizado



Aplicação

Banco de dados

Terminais

Fonte: autora, 2026

Cliente-Servidor: modelo centralizado

- Segundo Andrew S. Tanenbaum, em Computer Networks:
- *“Os primeiros sistemas computacionais compartilhados já demonstravam a necessidade de organizar recursos centralizados de maneira eficiente”.*

Fonte: technologyuk, 2026

Cliente-Servidor: histórico

E como já vimos, a criação da ARPANET (1969), surge a necessidade de:

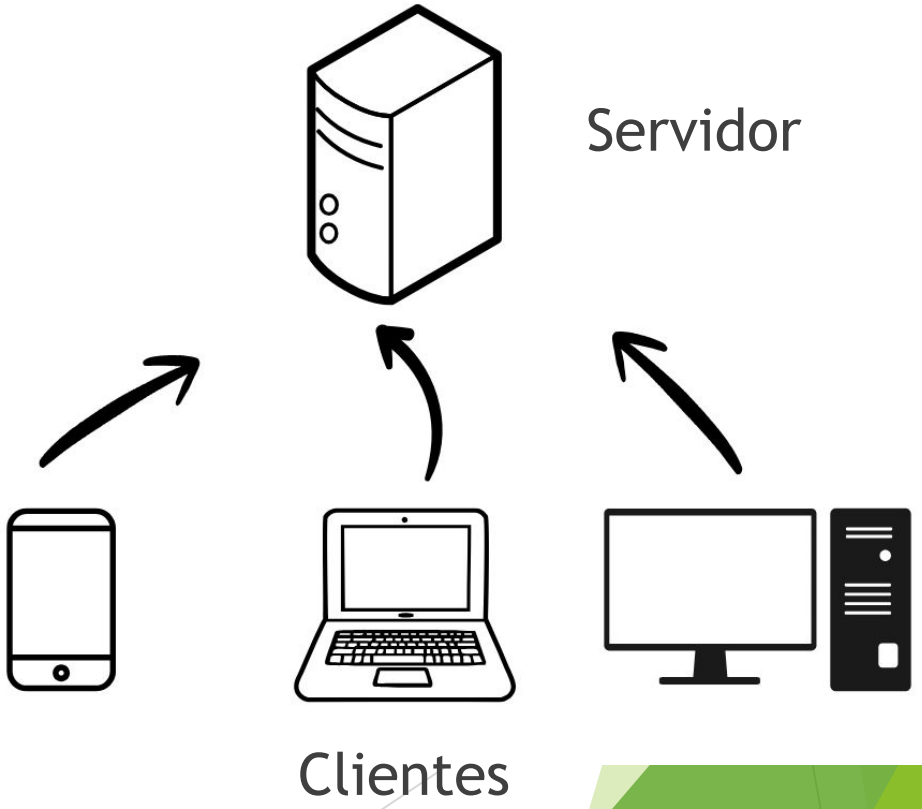
- Compartilhar recursos
- Acessar computadores remotamente
- Distribuir processamento

A popularização das redes e a necessidade de compartilhamento de recursos consolidaram o modelo cliente-servidor nas décadas de 1980 e 1990.

Cliente-Servidor: modelo distribuído

Segundo Andrew S. Tanenbaum (2011):

- O **modelo distribuído** caracteriza-se pela divisão de processos entre solicitantes de serviço (clientes) e provedores de serviço (servidores).



Fonte: autora, 2026

Cliente-Servidor: histórico

As principais **diferenças** do modelo centralizado x distribuído é:

- O terminal não executa lógica de aplicação localmente e apenas exibe o processamento do servidor
- Já o cliente compartilhar responsabilidades com o servidor fazendo alguns processamentos simples
- O terminal não tem autonomia nenhuma enquanto o cliente tem autonomia parcial

Cliente-Servidor: o que é cliente

Cliente:

- Inicia comunicação
- Solicita recursos ao servidor
- Faz execuções na máquina do usuário
- Processa interface gráfica
- Valida dados
- Executa parte da lógica da aplicação
- Pode armazenar dados temporariamente

Cliente-Servidor: exemplos de clientes

Exemplos de cliente:

- Um navegador fazendo requisição HTTP
- Um app mobile chamando uma API
- Um software desktop solicitando informações a um servidor.
- Um backend consumindo outra API
- Um script automatizado acessando um serviço
- Um microserviço chamando outro microserviço

Cliente-Servidor: exemplos de clientes

Exemplos famosos:

- Navegadores
- Spotify
- WhatsApp
- Microsoft Outlook

Fonte: autora, 2026

Cliente-Servidor: o que é cliente

Por que são clientes?

- Enviam requisições
- Recebem dados

Ou seja, seu funcionamento **depende** de um ou mais servidores.

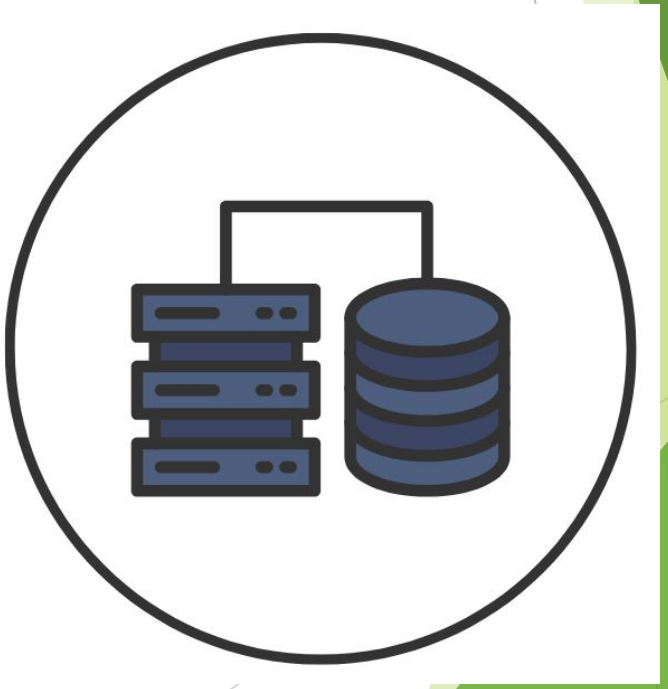
Mas ser cliente <u>não implica</u> ter necessariamente uma interface gráfica.

Cliente-Servidor: o que é servidor

- É um processo especializado que **disponibiliza serviços na rede**, permanecendo ativo e acessível para atender múltiplas requisições concorrentes de clientes, seguindo protocolos específicos de comunicação.

Principais características

- Fica aguardando requisições
- Fornece serviços ou recursos
- Pode atender múltiplos clientes simultaneamente



Fonte: autor, 2026

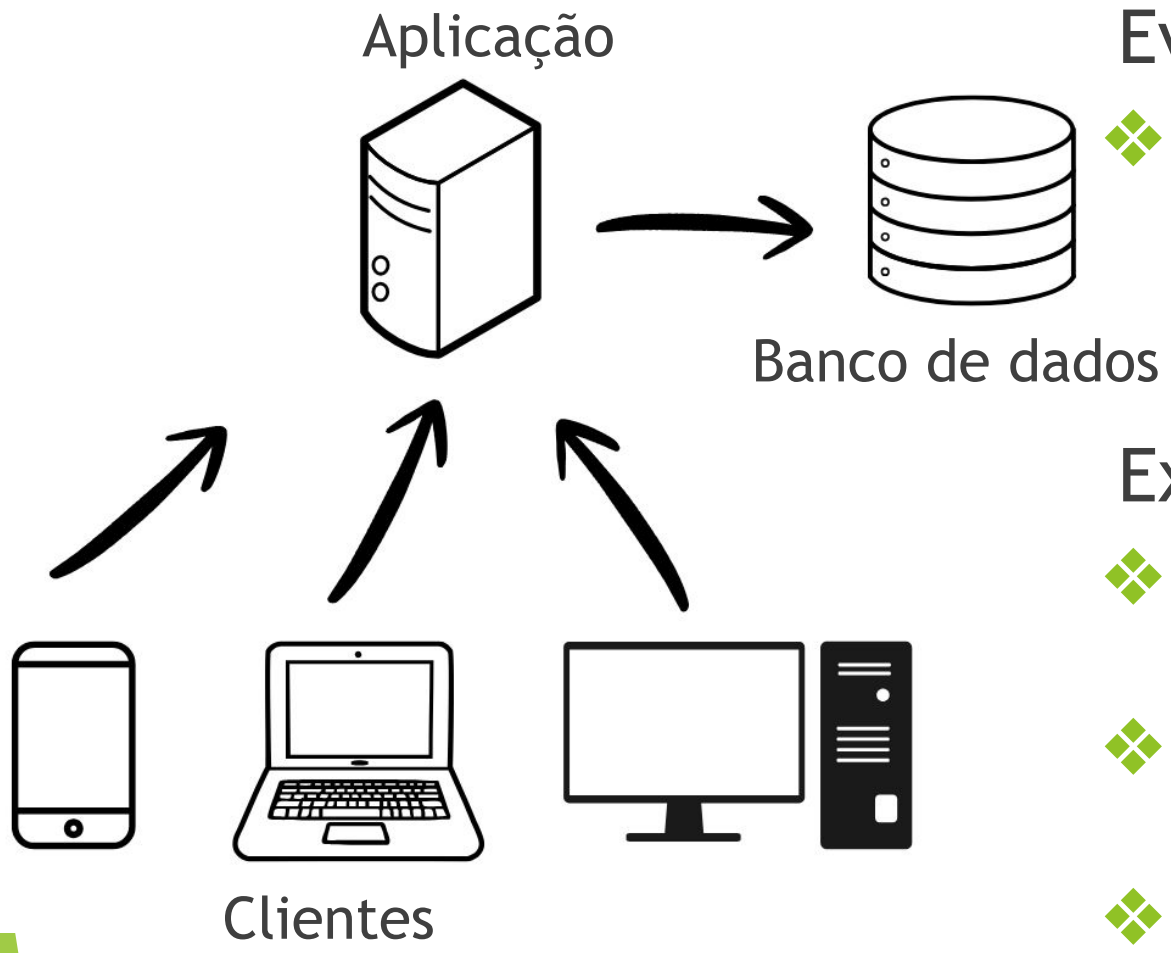
Cliente-Servidor: exemplos de servidores

Exemplos:

- Servidor Web (Apache, Nginx) (recebe requisições, armazena arquivos, encaminha requisições para aplicações)

- Banco de dados (Sistema de armazenamento que atende outros servidores)

Cliente-Servidor: arquitetura em camadas



Fonte: autora, 2026

Evolução da Arquitetura:

- Hoje temos estruturas baseadas em diversas camadas.

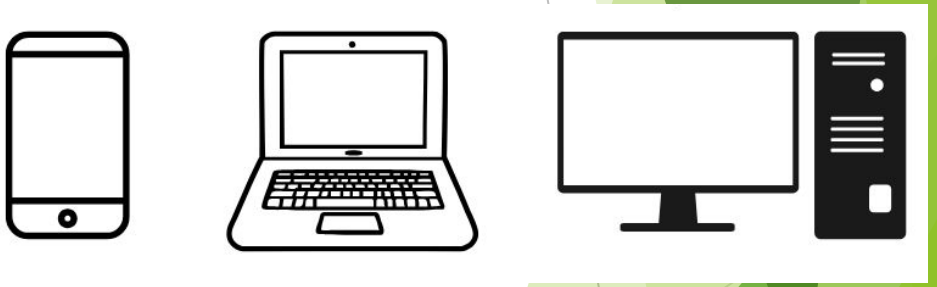
Exemplo clássico 3-tier:

- Camada de Apresentação
- Camada de Lógica de Negócio
- Camada de Dados

Cliente-Servidor: arquitetura em camadas

- O **cliente** é a aplicação que roda no dispositivo do usuário (navegador, app mobile, app desktop).
- O dispositivo (celular, notebook e computador) é apenas o meio físico.
- As aplicações web são **clientes** do servidor de aplicação, que guarda a regra de negócio e faz os processamentos necessários.
- E o servidor da aplicação, é **cliente** do banco de dados, que armazena as informações persistentes.

Meios físicos de acesso

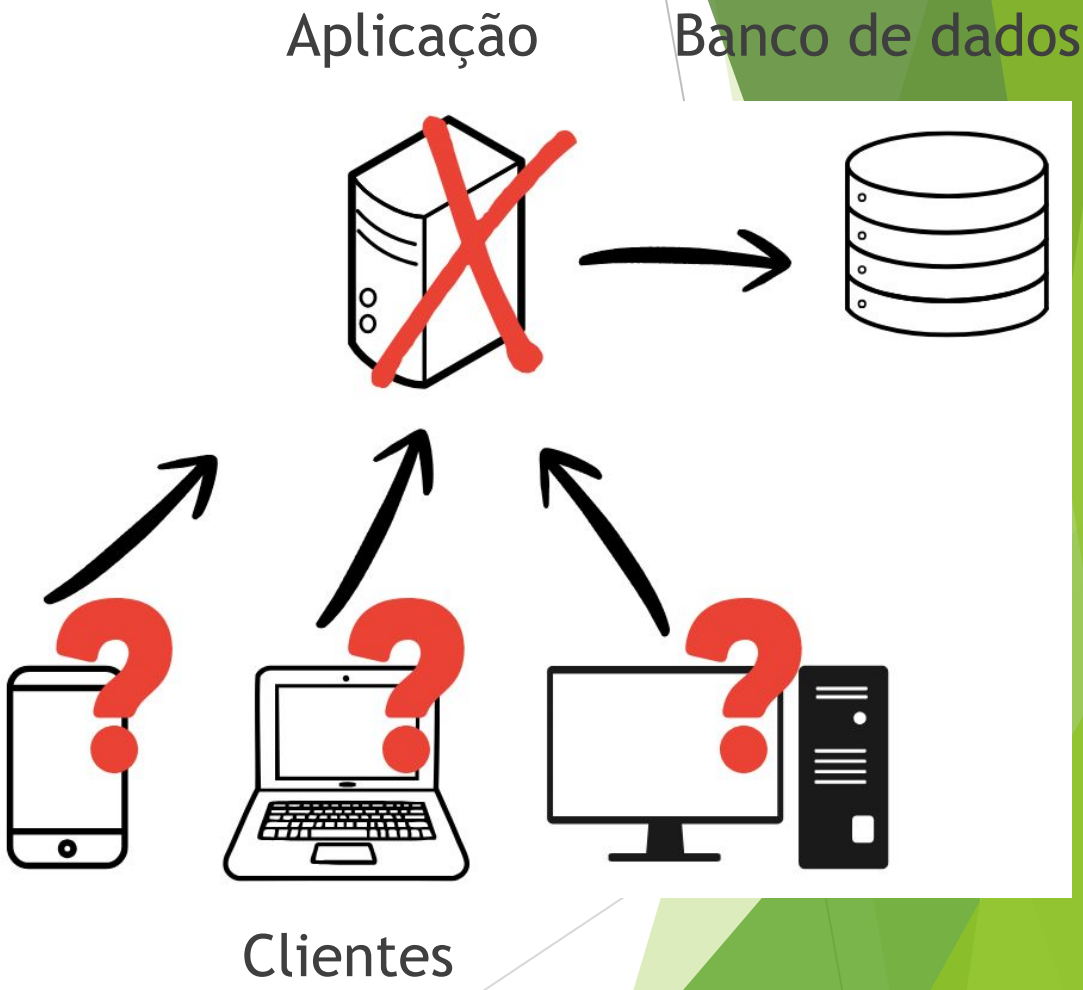


Fonte: autora, 2026

Cliente-Servidor: arquitetura em camadas

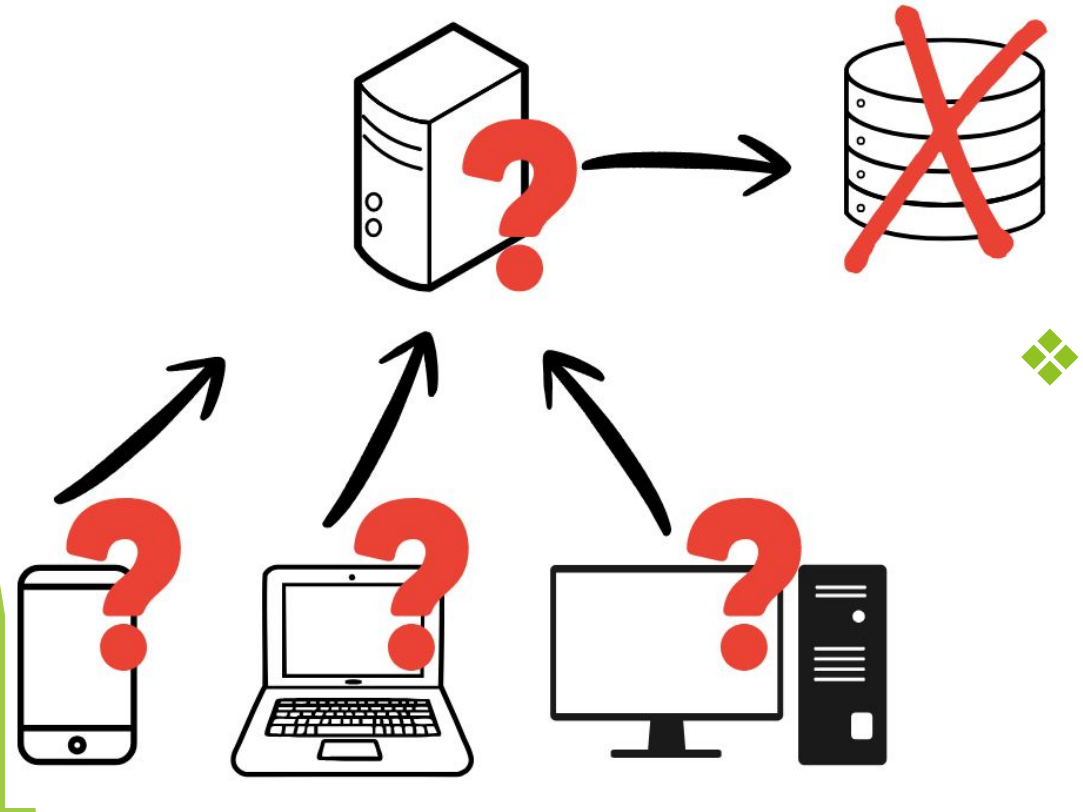
- Os **clientes** dependem das informações processadas enviadas pelos servidores para executar suas rotinas.

- Se o servidor falhar, o cliente será impactado.



Fonte: autora, 2026

Cliente-Servidor: arquitetura em camadas



Fonte: autora, 2026

- No modelo cliente-servidor existe dependência funcional.
- Se o servidor parar ou ficar indisponível ou responder com erro, <u>o cliente não consegue concluir sua operação</u>.

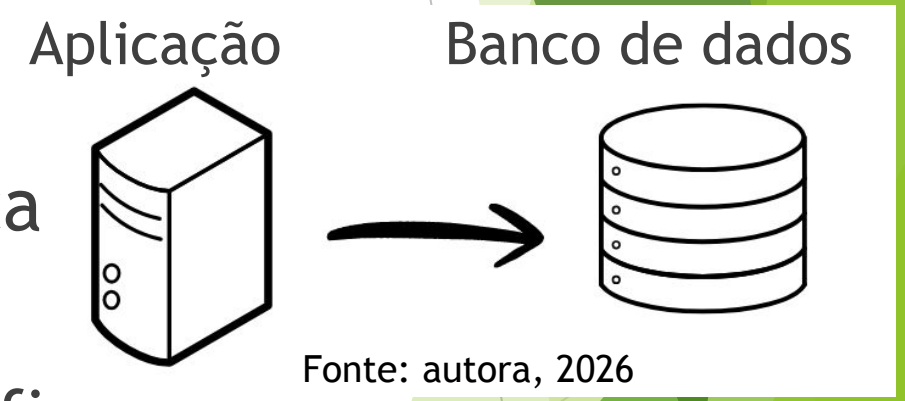
Cliente-Servidor: arquitetura em camadas

- O **servidor** segue funcionando de forma autônoma, aguardando aparecer clientes solicitando algo a eles.

Ele:

- Fica em execução contínua
- Escuta requisições em uma porta específica
- Responde sob demanda

O servidor não depende de um cliente específico para existir, mas depende de clientes para cumprir sua função.



Fonte: autora, 2026

Definição de rede x definição arquitetural

Cliente (Arquitetura de redes)

- Cliente é o processo que inicia uma requisição a um servidor.

Ou seja:

- Pode ser navegador
- Pode ser app mobile
- Pode ser software desktop
- Pode ser outro servidor

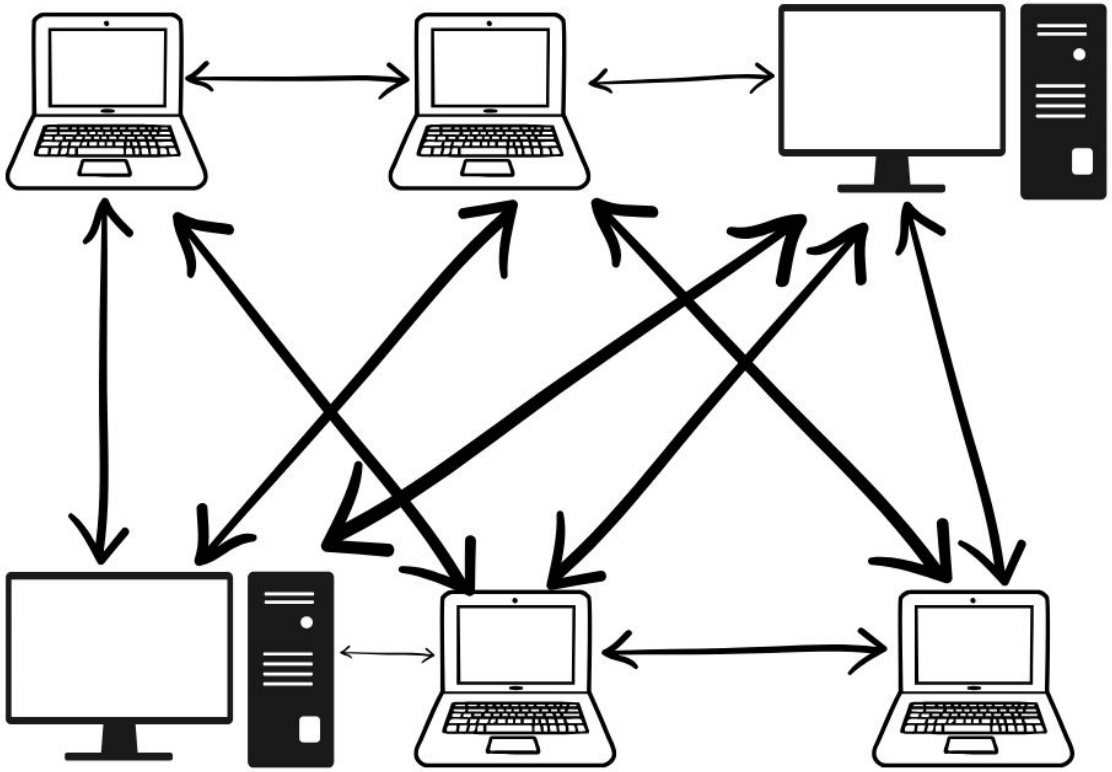
Cliente é um papel na comunicação, não um tipo de interface.

Frontend (Arquitetura de software)

Frontend é:

- A camada de interface com o usuário
- Responsável pela apresentação
- Parte da aplicação que roda no dispositivo do usuário
- Ele geralmente é cliente, o frontend de uma aplicação web é um cliente.

Arquitetura Peer-to-Peer (P2P)



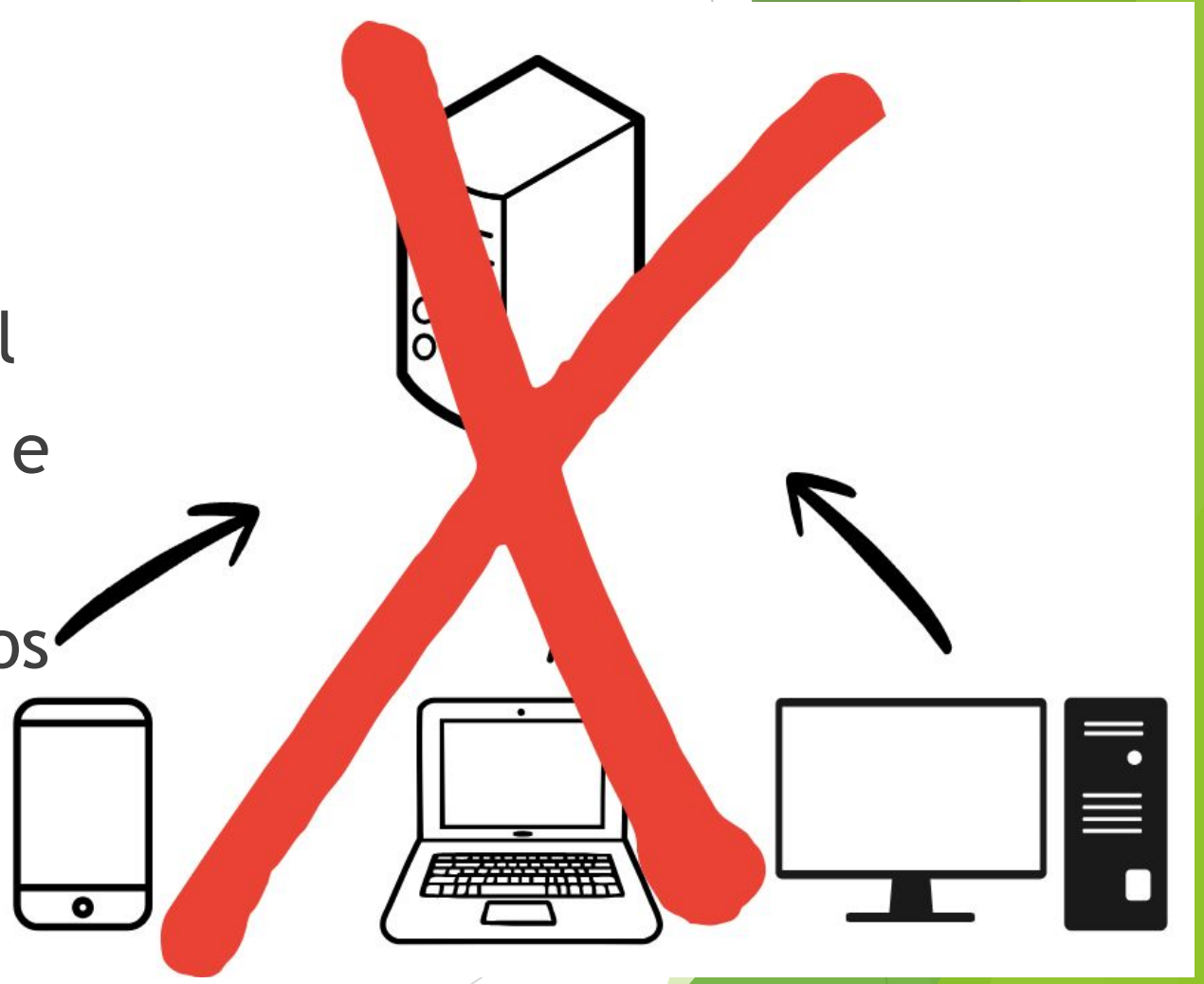
Fonte: autora, 2026

No modelo peer-to-peer, todos os **nós** são chamados de **peers** (pares) que podem atuar tanto como cliente quanto como servidor ao mesmo tempo.

Arquitetura Peer-to-Peer (P2P)

Ou seja:

- Não existe um servidor central
- Cada máquina pode receber e fornecer recursos para outras
- O tráfego é distribuído entre os pares



Fonte: autora, 2026

Arquitetura Peer-to-Peer (P2P)

Características:

- Descentralização (sem ponto único de falha)
- Escalabilidade natural, quanto mais peers, mais recursos
- Utilização eficiente de banda e armazenamento

Desafios:

- Controle e segurança mais complexos
- Coordenação dos nós exige protocolos eficientes

Arquitetura Peer-to-Peer (P2P)

Fonte: autora, 2026

- BitTorrent é um **protocolo de comunicação P2P** criado por Bram Cohen para distribuir dados de forma eficiente.

- Softwares clientes que seguem esse protocolo permitem que cada participante atue simultaneamente como cliente e servidor, compartilhando partes do arquivo com outros peers, sem depender de um servidor central para a distribuição dos dados.

Arquitetura Peer-to-Peer (P2P)

Protocolo = conjunto de regras que define:

- como os peers se encontram
- como os arquivos são divididos
- como os blocos são trocados
- como a integridade é verificada

Ele **implementa** arquitetura P2P, mas ele não é a arquitetura em si.

- Arquitetura P2P é um modelo conceitual.
- BitTorrent é uma implementação concreta desse modelo.

Arquitetura Peer-to-Peer (P2P)

Fonte: autora, 2026

Arquitetura Peer-to-Peer (P2P)

1 Fragmentação do arquivo

- O arquivo a ser compartilhado é dividido em vários pedaços (chunks ou pieces).

Isso é essencial porque:

- Permite download simultâneo de múltiplos peers.
- Permite que um peer compartilhe mesmo antes de terminar o download.

2 Entrada na rede (Swarm)

- O conjunto de peers que compartilham um mesmo arquivo é chamado de swarm.

Dentro do swarm existem:

- Seeder: Peer que possui o arquivo completo.
- Leecher: Peer que ainda está baixando.

Arquitetura Peer-to-Peer (P2P)

3 Comunicação entre peers

Ao entrar na rede:

- O peer descobre outros peers (via tracker ou DHT).
- Solicita pedaços diferentes para diferentes peers.
- Envia pedaços para quem solicitar.

Isso gera um efeito muito importante:

- Quanto mais pessoas baixando, mais rápida tende a ser a distribuição.
- Diferente do modelo cliente-servidor, onde muitos usuários sobrecarregam um único servidor.

Arquitetura Peer-to-Peer (P2P)

Clientes BitTorrent (softwares):

- qBittorrent
- uTorrent

Esses são programas que utilizam o protocolo.

Fonte: autora, 2026

Cliente-Servidor x Peer-to-Peer

No modelo cliente-servidor:

- Papéis bem definidos
- Um servidor central armazena tudo
- Todos os clientes dependem dele

No P2P:

- Todos podem ser cliente e servidor
- Não existe servidor central para os dados
- Os próprios usuários distribuem os arquivos

Dúvidas?

Atividade 2

- Responder questionário referente aos assuntos da aula de hoje.
- **Data de entrega**: HOJE até às 22h30min
- **Forma de entrega**: manuscrito físico.
- **Execução**: duplas (colocar nome de todos)
- **Fonte de consulta**: material da aula, sua dupla e internet.

Referência da aula

- TECHNOLOGYUK. Mainframe computers. TechnologyUK.net, s.d. Disponível em: https://www.technologyuk.net/computing/computer-hardware/mainframe-computers.shtml. Acesso em: 26 fev. 2026.

- TANENBAUM, Andrew S.; WETHERALL, David J. Computer Networks. 5. ed. Boston: Pearson, 2011.

- KUROSE, James F.; ROSS, Keith W. Computer Networking: A Top-Down Approach. 7. ed. Boston: Pearson, 2017.

- ANDERSON, Ross. Security Engineering: A Guide to Building Dependable Distributed Systems. 2. ed. Indianapolis: Wiley, 2008.

Referência da aula

- MOCKAPETRIS, Paul. Domain names – concepts and facilities. RFC 1034. 1987.
- MOCKAPETRIS, Paul. Domain names – implementation and specification. RFC 1035. 1987.
- ICANN. Internet Corporation for Assigned Names and Numbers. Disponível em: https://www.icann.org
- NIC.BR. Núcleo de Informação e Coordenação do Ponto BR. Disponível em: https://www.nic.br

univille

univille.br